

# Descripción de la Persistencia de Datos

Proyecto: Sanos y Salvos | Asignatura: DSY1106 Desarrollo Fullstack III | Evaluación: Parcial N°3 | Fecha: Junio 2026

## 1. Tecnologías de Persistencia Utilizadas

| Tecnología                  | Uso                                    | Servicio                                    |
|-----------------------------|----------------------------------------|---------------------------------------------|
| PostgreSQL + JPA/Hibernate  | Persistencia principal de entidades    | ms-mascotas, ms-coincidencias, auth-service |
| PostGIS + Hibernate Spatial | Persistencia y consultas geoespaciales | ms-geolocalizacion                          |
| Redis                       | Caché de sesiones y dashboard          | auth-service, bff-service                   |
| MinIO (S3-compatible)       | Almacenamiento de fotos de mascotas    | ms-mascotas                                 |

## 2. JPA / Hibernate — Persistencia Relacional

### 2.1 Configuración Base

Todos los microservicios con BD relacional usan **Spring Data JPA** con Hibernate como implementación ORM. La configuración se declara en `application.yml`:

```
spring:
  datasource:
    url: jdbc:postgresql://postgres:5432/mascotas_db
    username: ${DB_USER}
    password: ${DB_PASS}
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
```

### 2.2 ms-mascotas — Modelo de Entidades

El microservicio usa el **patrón Factory Method** para crear reportes y **herencia de tabla única** en JPA:

Reporte (clase base)

└ ReportePerdido → campo: recompensa

└ ReporteEncontrado → campos: lugarResguardo, tieneCollar

Entidad principal:

```

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "tipo")
public abstract class Reporte {
    @Id @GeneratedValue
    private UUID id;

    private String especie;
    private String raza;
    private String nombre;
    private String color;
    @Enumerated(EnumType.STRING)
    private Tamano tamano; // PEQUEÑO, MEDIANO, GRANDE
    private Double lat;
    private Double lng;
    private String direccion;
    private String fotoUrl;

    @Enumerated(EnumType.STRING)
    private EstadoReporte estado; // ACTIVO, RESUELTO, ELIMINADO

    @CreationTimestamp
    private LocalDateTime createdAt;
}

```

#### Repositorio JPA:

```

public interface ReporteRepository extends JpaRepository<Reporte, UUID> {
    @Query("SELECT r FROM Reporte r WHERE r.estado = 'ACTIVO'")
    List<Reporte> findAllActivos();

    List<Reporte> findByUserId(UUID userId);
}

```

### 2.3 ms-coincidencias — Modelo de Entidades

```

@Entity
public class Coincidencia {
    @Id @GeneratedValue
    private UUID id;

    private UUID reportePerdidoId;
    private UUID reporteEncontradoId;
    private Double scoreTotal; // 0-100
    private Double scoreRaza;
    private Double scoreTamano;
    private Double scoreColor;
    private Double scoreGeo;
    private Double distanciaKm;

    @Enumerated(EnumType.STRING)
    private EstadoCoincidencia estado; // PENDIENTE, CONFIRMADA, DESCARTADA
}

```

### 2.4 auth-service — Modelo de Entidades

```

@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue
    private UUID id;

    private String nombre;
    @Column(unique = true)
    private String email;
    private String password; // BCrypt encoded

    @Enumerated(EnumType.STRING)
    private RolUsuario rol; // USER, REFUGIO, ADMIN

    private boolean emailVerificado;
    private String tokenVerificacion;
}

```

## 3. PostGIS — Persistencia Geoespacial

### 3.1 Extensión PostgreSQL

**ms-geolocalizacion** usa **PostGIS** para almacenar y consultar coordenadas geográficas con precisión espacial:

```

-- Habilitado en el init script
CREATE EXTENSION IF NOT EXISTS postgis;

```

### 3.2 Entidad con Campo Geométrico

```

@Entity
public class PuntoGeo {
    @Id @GeneratedValue
    private UUID id;

    private UUID reporteId;
    private String tipo; // PERDIDO | ENCONTRADO

    @Column(columnDefinition = "geometry(Point,4326)")
    private Point ubicacion; // org.locationtech.jts.geom.Point
}

```

### 3.3 Consulta Espacial con Hibernate Spatial

```

// Buscar puntos dentro de un radio (km) usando ST_DWithin de PostGIS
@Query(value = ""
    SELECT * FROM punto_geo
    WHERE ST_DWithin(
        ubicacion::geography,
        ST_SetSRID(ST_MakePoint(:lng, :lat), 4326)::geography,
        :radiusMeters
    )
    """, nativeQuery = true)
List<PuntoGeo> findNearby(double lat, double lng, double radiusMeters);

```

## 4. Redis — Caché

### 4.1 auth-service (Caché de Sesiones)

Redis almacena tokens de refresco y control de rate limiting:

```

@Service
public class RateLimitService {
    private final StringRedisTemplate redis;

    public boolean isRateLimited(String ip) {
        String key = "rate:" + ip;
        Long count = redis.opsForValue().increment(key);
        if (count == 1) redis.expire(key, Duration.ofMinutes(1));
        return count > 10; // máximo 10 intentos por minuto
    }
}

```

## 4.2 bff-service (Caché de Dashboard)

El BFF cachea el endpoint `/api/dashboard` para evitar consultar todos los microservicios en cada petición:

```

@Service
public class DashboardService {
    private static final String CACHE_KEY = "dashboard:data";
    private static final long TTL_SECONDS = 300; // 5 minutos

    public Map<String, Object> getDashboard() {
        String cached = redis.opsForValue().get(CACHE_KEY);
        if (cached != null) return objectMapper.readValue(cached, Map.class);

        // Cache MISS: consultar microservicios
        Map<String, Object> data = buildDashboard();
        redis.opsForValue().set(CACHE_KEY, objectMapper.writeValueAsString(data),
            Duration.ofSeconds(TTL_SECONDS));
        return data;
    }

    public void invalidateCache() {
        redis.delete(CACHE_KEY);
    }
}

```

## 5. MinIO — Almacenamiento de Fotos

MinIO es un servidor de objetos compatible con S3 que almacena las fotos de los reportes:

```

@Service
public class MinioService {
    private final MinioClient minioClient;

    public String uploadImage(MultipartFile file, UUID reporteId) {
        String objectName = reporteId.toString() + ".jpg";
        minioClient.putObject(PutObjectArgs.builder()
            .bucket("mascotas-fotos")
            .object(objectName)
            .stream(file.getInputStream(), file.getSize(), -1)
            .contentType(file.getContentType())
            .build());
        return minioClient.getPresignedObjectUrl(...); // URL pública
    }
}

```

### Configuración:

```

minio:
  endpoint: http://minio:9000
  access-key: ${MINIO_ACCESS_KEY}
  secret-key: ${MINIO_SECRET_KEY}
  bucket: mascotas-fotos

```

## 6. Diagrama de Flujo de Persistencia



## 7. Garantías de Integridad

| Mecanismo             | Implementación                                              |
|-----------------------|-------------------------------------------------------------|
| Transacciones JPA     | @Transactional en métodos de servicio                       |
| Validación de entrada | @Valid + @NotBlank, @Email en DTOs                          |
| Unicidad de email     | @Column(unique = true) en entidad User                      |
| Estado de reporte     | @Enumerated con valores fijos (ACTIVO, RESUELTO, ELIMINADO) |
| Contraseñas           | BCrypt hash (PasswordEncoder) — nunca texto plano           |
| Caché coherente       | invalidateCache() tras cada CREATE/UPDATE de reporte        |